Hands-on Exercise: Build a LangChain AI Agent with Memory, Tools and Guardrails

Duration: 2 Hours
Technology: Python, PyCharm, LangChain, OpenAI GPT-4o-mini
Difficulty: Beginner

Problem Statement

Build an **AI Restaurant Reservation Assistant** using LangChain.

The agent should help customers explore available restaurants, calculate estimated dining costs, and remember information from earlier conversations.

The application must use **LangChain agents, two Python tools, conversation memory and basic guardrails**.

No frontend or database is required.

Task 1: Create the Python Project

Create a project with the following structure:

```
Restaurant_AI_Agent/
├── agent.py
├── tools.py
├── guardrails.py
├── restaurants.json
├── requirements.txt
└── .env
```

Install the required Python libraries using `requirements.txt` and configure your OpenAI API key in `.env`.

Task 2: Prepare Sample Data

Create `restaurants.json` containing at least five fictional restaurants.

Each restaurant should include:

- Restaurant ID
- Restaurant name
- Cuisine type
- Location
- Average price per person
- Seating availability

Example:

```
{
  "id": "R101",
  "name": "Spice Garden",
  "cuisine": "Indian",
  "location": "Hyderabad",
  "price_per_person": 800,
  "available_seats": 20
}
```

Task 3: Create Two LangChain Tools

Use the `@tool` decorator.

Tool 1: `search_restaurants()`

- Accept a cuisine type or restaurant keyword.
- Search the restaurant JSON file.
- Display matching restaurants with prices and available seats.

Tool 2: `calculate_dining_cost()`

- Accept restaurant ID and number of guests.
- Retrieve the price per person.
- Calculate and return the estimated total dining cost.
- Return an appropriate message if the restaurant doesn't exist.

Do not implement actual reservation bookings or payments.

Task 4: Create the LangChain Agent

Using `create_agent()` and GPT-4o-mini:

1. Register both Python tools with the agent.
2. Let the model choose which tool to execute.
3. Execute the selected tool.
4. Return the tool result to the model.
5. Display the final AI-generated response.
6. Print the selected tool name and arguments in the terminal.

Task 5: Implement Conversation Memory

Use LangChain's `InMemorySaver` to maintain conversation history.

The agent must remember:

- Previously selected restaurant
- Number of guests
- Previous dining-cost calculations

Provide a `/new` command to start a fresh conversation.

Test conversation

User: Show me Indian restaurants.

Agent: Displays matching restaurants.

User: I want Spice Garden for 4 people. What's the estimated cost?

Agent: Calculates the cost for 4 people.

User: What if I increase it to 6 people?

Agent: Recalculates using the previously selected restaurant.

Task 6: Implement Basic Guardrails

Create `guardrails.py` and add these protections:

Guardrail	Requirement
Domain restriction	Allow only restaurant-related questions

Guardrail	Requirement
Input validation	Reject empty or excessively long questions
Context handling	Allow follow-up messages such as "yes", "no", or "6 people"
Tool validation	Reject guest counts less than 1 or greater than 20
PII detection	Detect email addresses and phone numbers
Output guardrail	Redact detected email addresses and phone numbers

Ensure the system prompt also instructs the agent not to invent restaurant names, prices or availability.

Task 7: Test the Agent

Test the following scenarios:

1. "Show me Indian restaurants."
2. "What is the estimated cost for 4 people at Spice Garden?"
3. "Make that 6 people."
4. "Show me Italian restaurants."
5. "Calculate the cost for 50 guests."
6. "What is the largest mammal?"
7. "My email is john@example.com."
8. Type `/new` and verify that previous conversation details are no longer available.

Expected Outcome

The completed application should demonstrate:

- LangChain agent creation with GPT-4o-mini
- Two Python tools using `@tool`
- Model-driven tool selection and execution
- Conversation memory using `InMemorySaver`
- Contextual follow-up handling
- Domain-restricted interactions
- Input, tool and output guardrails

Submission Requirements

Submit:

1. Complete Python project with all source files.
2. `restaurants.json` containing at least five restaurants.
3. Screenshots showing both tools executing.
4. Screenshot demonstrating conversation memory.
5. Screenshot showing an unrelated question being blocked.
6. Screenshot demonstrating guest-count validation and PII detection.

Final Objective: Build a simple, functional LangChain AI agent that selects tools, remembers conversations and applies guardrails before executing actions or displaying responses.